

JVD Internal Management System

Operations, Training & Rollout Guide · v1.0 FINAL · 2026-07-04

JVD Operations, Training & Rollout Guide

Date: 2026-07-04 · Final document of the v1.0 set (see `README.md` for reading order) Covers the remaining topics: Philippines deployment choices, caching, maintenance & IT support, dependency/security hygiene, PH compliance, system handoff, and the employee training/rollout plan.

1. Deployment — concrete choices from the Philippines

Your users are PH-based office staff, drivers on phones, and customers opening portal links. The deciding factor is latency to a reliable region + operational simplicity.

Recommended setup (Tier 1):

Piece	Choice	~Cost/mo
Production server	One VPS in Singapore (DigitalOcean SGP1, Vultr Singapore, or AWS Lightsail SG) — 4 vCPU / 8 GB, Docker Compose runs app + queue worker + Reverb	\$40–60
Database	Postgres in the same Compose stack to start; move to managed Postgres (DO Managed DB) when revenue justifies it	\$0 → +\$15–30
Staging	One small droplet (2 GB), same region, same Docker images	\$12–18
Files + backups	S3-compatible object storage (DO Spaces / Backblaze B2) — uploads via Laravel's s3 disk + nightly <code>pg_dump</code> pushed here	\$5–10
CDN / DNS / TLS	Cloudflare free tier in front (caches Vite's hashed assets, free SSL, DDoS shielding)	\$0
Email delivery	Resend, Postmark, or SES (never the VPS's own SMTP — deliverability)	\$0–15
Total		~\$60–100/mo

Why Singapore, not a PH datacenter: 25–50 ms from PLDT/Globe/Converge — imperceptible — while pricing, tooling, and reliability of SG regions beat local hosting. Why not shared/cPanel PH hosting: it can't run queue workers, Reverb websockets, or Docker — your architecture requires a real server.

Tier 2 (less DevOps learning): Laravel Forge (\$12/mo) managing the same DO droplet — it handles nginx, PHP, queue workers, SSL, and zero-downtime deploys via a UI, and you still learn the concepts. A good middle path if Docker feels heavy early on; you can migrate to raw Docker later.

PH-specific notes: - Power/ISP interruptions are a fact of life → this is why offsite backups (\$1 table) and uptime alerts (Phase 1) are non-negotiable, and why the office should not host anything. - Payments: you already use PayMongo (GCash/cards) — correct local choice; keep it. - **BIR awareness:** once the system prints official receipts/invoices as your books of record, look into BIR Computerized Accounting System (CAS)

registration requirements with your accountant. Flagging it now so it's a planned conversation, not a surprise audit finding. - **Data Privacy Act (RA 10173)**: you store passports, IDs, and KYC documents — you are a personal-data processor under NPC rules. Minimum bar: appoint a Data Protection Officer (can be an existing officer), register with the NPC if thresholds apply, add a privacy notice to the customer portal and KYC forms, define retention (§4 DMS `retention_until` field already supports this), and have a breach-notification plan (NPC: 72 hours). The Phase 2 security items (private storage, access control, audit logs) are most of the technical compliance already.

2. Caching — what to cache, how, and what never to cache

Caching has exactly one professional rule: **cache derived, read-heavy data with a clear invalidation story — never cache anything on a money-mutation path.**

The four layers, in order of payoff:

1. **Deploy-time caches (free, do first)**: `php artisan config:cache route:cache view:cache event:cache` in the deploy script. Instant framework speedup.
2. **Application cache (Redis — already configured, unused)**: `php // Dashboard widgets — 60s is invisible to users, kills repeated heavy queries Cache::remember("widget:revenue:{<code>$month</code>}", 60, fn () => /* heavy query */); // Reports — 10 min Cache::remember("report:aging:{<code>$params</code>}", 600, fn () => ...); // System settings & role permissions — until changed (invalidate on write) Cache::rememberForever('settings:public', fn () => ...); // Cache::forget on update Cache::remember("perms:user:{<code>$id</code>}", 300, ...); // bust on role change Invalidation is event-driven: the model observer/listener that changes the data calls Cache::forget(...) — never rely on TTL alone for things users just edited.`
3. **Client cache (React Query — Phase 3.4)**: `staleTime: 30_000` for lists, `staleTime: Infinity` + explicit invalidation for reference data (categories, buses, roles). This is what makes the UI feel instant.
4. **CDN (Cloudflare)**: caches hashed JS/CSS/images forever automatically. Zero config beyond turning it on.

Never cache: availability checks (bus/driver — must hit the DB with locks), invoice/payment reads inside a transaction, anything the user is about to edit, permission checks on financial actions (5-min perms cache is fine for *menus*; re-check live before *approving*).

Key convention: `{domain}:{entity}:{qualifier}` (`widget:collections-due:2026-07`, `perms:user:42`) — greppable, pattern-deletable.

3. Maintenance & IT support — don't build a second system

Should you build a separate helpdesk/maintenance system? **No — and this is a professional-restraint decision.** A bespoke ticketing module is another vibe-coded system to maintain. Instead:

1. **Dogfood the workflow engine (Spec §5)**: create an "IT Request" workflow (report → triage → resolved). It's a category + steps in the engine you're already building — near-zero extra code, and requests appear in the same approvals/tasks inbox users already know.
2. **"Report a problem" button** in the AppShell (Design's Help Center slot): opens a small modal, auto-attaches page URL, user, role, and browser info to the request. That context is 80% of IT triage time.

3. **FAQs & guides live in the system:** a Help Center page rendering markdown articles (stored in the repo `docs/help/` , versioned with the code so they update with each release). Start with the 20 questions the pilot group actually asks — not 100 guesses.
4. **Sentry (Phase 1) is the other half of IT support** — errors report themselves with stack traces before users even complain.

Routine maintenance calendar (goes in the runbook):

Cadence	Task
Daily (automated)	Backups + uptime checks + failed-jobs alert
Weekly	Review Sentry issues, check disk/queue depth, review slow-query log
Monthly	<code>composer audit</code> + <code>npm audit</code> and patch (today's findings: <code>guzzle/psr7</code> — 3 medium CVEs, <code>composer update guzzlehttp/guzzle guzzlehttp/psr7</code> ; <code>exceljs→uuid</code> — 2 moderate, <code>npm audit fix</code>). Add both audits to CI so they're never forgotten.
Quarterly	Restore-test a backup, review user accounts (offboarding leaks), dependency minor upgrades
Yearly	PHP/Laravel/Node major-version review, NPC/DPA compliance review

4. Handoff & turnover — the bus-factor kit

Everything a new IT hire (or a departing dev's replacement) needs, kept current as part of "done":

1. **This doc set** (`docs/README.md` reading order) — architecture, decisions, roadmap.
2. **Runbook** (`docs/RUNBOOK.md` , write in Phase 1): how to deploy, roll back, restart services, read logs, restore a backup, rotate credentials — each as numbered steps tested by someone who didn't write them.
3. **Access inventory:** every third-party account (hosting, Cloudflare, PayMongo, email provider, Sentry, GitHub org, domain registrar) with who holds admin — stored in a shared password manager (Bitwarden/1Password), **never** in chat logs or spreadsheets.
4. **Environment inventory:** every `.env` key documented (what it does, where its value lives).
5. **Two-person rule:** at least two people can deploy and restore. Rotate who does the monthly maintenance so knowledge stays spread.
6. **Employee on/offboarding checklist:** create user + role + 2FA enrollment on day one; disable account + revoke sessions + reassign open workflow items on exit (the abilities system §6 makes "reassign approver" a data change).

5. Training & rollout — how to launch this to the company

Rollout strategy: pilot → champions → everyone, module by module (matches the Phase 3 migration order — you're never training on software that's about to change):

1. **Pilot (per module):** 2–3 real users work in **staging** with seeded data for a week. Their confusion rewrites your FAQs; their bugs precede everyone else's.
2. **Champions:** one power user per department, trained first and named in the Help Center. People ask the deskmate before IT — make that official.

3. **Go-live per module** with a 2-week feedback window (the "Report a problem" button + a group chat channel).
4. **Measure adoption:** logins per role, workflows completed in-system vs on paper. Paper persisting = a training gap or a UX gap; find out which.

In-app tutorials — yes, and the design already contains them. The Dashboard Flaws Figma file's onboarding frames are exactly this pattern, and they're already specced as components (Design §2 #11–12):

- **First-login checklist** (OnboardingChecklist component): role-specific, 4–6 real tasks — agent: "Create your first quotation → Add a customer → Check your approvals inbox", with progress bar ("4 of 6 complete"). Anchored to real work, dismissible, reopenable from the Help Center.
- **Spotlight tooltips** (FeatureCallout component): small dismissible cards pointing at 3–5 key controls the first time a page is visited ("Approvals wait here", "⌘K searches everything"). Per-page, per-user `seen` flags.
- **"New feature" popovers** after each phase ships — the changelog card from the Figma file. This is how you introduce the redesign gradually instead of shocking everyone at once.
- **Rules that keep tours helpful:** never more than ~5 steps, always skippable, task-anchored (point at what they're doing, not a 20-screen abstract walkthrough), and never block the UI.

Live training: one hands-on hour per role group in staging (not a lecture — they do the tasks from their checklist), recorded as 3–5-minute task videos ("How to create a bus rental booking") linked from the Help Center. Short task videos beat a manual nobody reads.

Roadmap insertion: these land as **Phase 4.5 — Rollout** (after testing, before/alongside Phase 5): help center + FAQs seeded from pilot, onboarding checklists per role, champion training, module go-lives with feedback windows.

6. Anything else left to look into? (the honest completeness check)

Areas reviewed and deliberately summarized rather than given full documents — revisit each at its natural moment:

- **Accessibility (Phase 3):** build into the component library once — keyboard navigation and focus traps in Modal/Drawer, visible focus rings, WCAG AA contrast (the §1 palette passes), labels on all inputs, `aria-live` on toasts. Free if built into components; expensive if retrofitted per page.
- **Mobile/responsive (Phase 3):** drivers and field staff use phones — DataTable needs a card-collapse mode at small widths; the driver dashboard is the mobile-first pilot.
- **PDF/report quality (Phase 3):** consolidate on backend dompdf with one letterhead template (the tri-color brand line lives here) instead of frontend jsPDF drift.
- **Seeded/demo data (Phase 4):** a realistic `DemoSeeder` for staging/training — believable customers, a year of transactions. Training quality depends on it.
- **Analytics/BI (post-Phase 5):** once the ledger is the money truth, reporting (or Metabase pointed at a read replica) becomes trivial. Don't build BI before the ledger exists.

With this document, the planning set is complete — the remaining unknowns are ones that fieldwork (workshops, pilots, load tests) must answer, not more analysis.